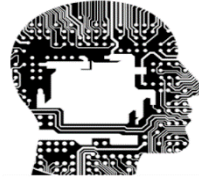




Université Constantine 2
جامعة قسنطينة 2



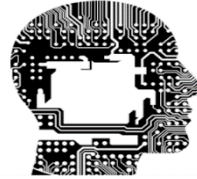
Apprentissage par Machine 02

APM2

Dr. NECIBI Khaled
New Technologies Faculty
Khaled.necibi@univ-constantine2.dz



Université Constantine 2
جامعة قسنطينة 2



Apprentissage par Machine APM 02

Workshop 02

Dr. NECIBI Khaled

New Technologies Faculty

Khaled.necibi@univ-constantine2.dz

Etudiants concernés

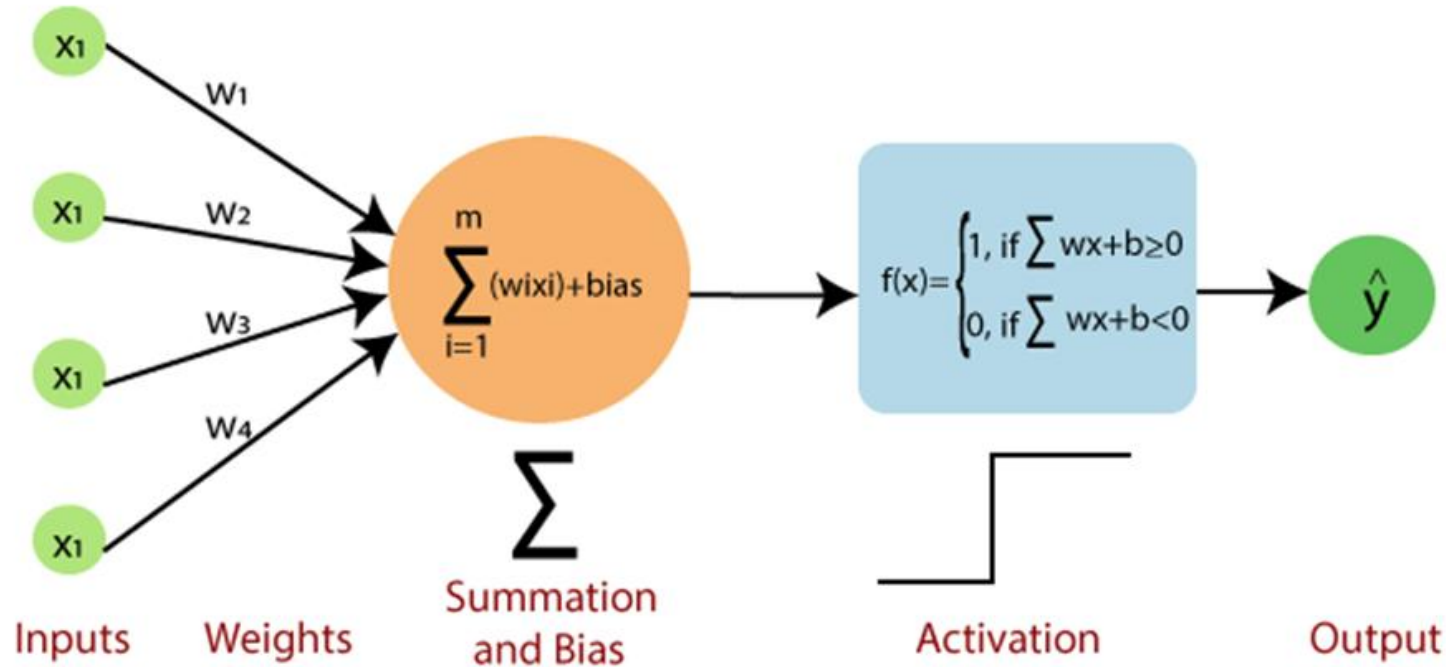
Faculté/Institut	Département	Niveau	Spécialité
New Technologies	IFA	Master 01	STIC

Simple Neural Network Architecture

● Single Layer Perceptron (SLP)

- A Single-Layer Perceptron is the simplest type of neural network, consisting of:
 - Input layer with **features**: accepts multiple features $(x_1, x_2, \dots, x_n)$
 - **Weights**: $(w_1, w_2, \dots, w_n)$ adjust inputs contributions
 - **Bias**: b shifts the decision boundaries
 - **Activation function**: determines the final decision
- The perceptron follows this equation:
- $z = \sum_{i=1}^n w_i x_i + b$ where:
 - z is the weighted sum of inputs,
 - w_i are weights,
 - x_i are the input values, and
 - b is the bias
- The final output $\hat{y}$ is determined by applying an activation function $f(z)$

Simple Neural Network Architecture



Simple Neural Network Architecture

● Applications of SLP

- SLP is primarily used for linearly separable classification problems
- Email Spam Detection
- Handwritten Digit Recognition
- Medical Diagnosis (binary classification)
- Image classification ( vs. )
- Sentiment Analysis (binary text classification)
- Face Recognition (match vs. not match)
- Works well for binary classification tasks!
- Effective for linearly separable problems

Simple Neural Network Architecture

- Most common activation functions in SLP

- Step Function (Threshold function)

- $$f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- Use case: binary classification (0 or 1)

- Limitation: can not handle continues outputs or probabilistic interpretations

- Sign Function

- $$f(z) = \begin{cases} 1, & \text{if } z > 0 \\ -1, & \text{if } z \leq 0 \end{cases}$$

- Use case: classification with labels -1 and 1 instead of 0 and 1

- Limitation: similar to the step function, no probability output

Simple Neural Network Architecture

- Most common activation functions in SLP
 - Sigmoid Function
 - $f(z) = \frac{1}{1+e^{-z}}$
 - Use case: When probability estimation is needed
 - Limitation: Computationally expensive and prone to vanishing gradients
 - Hard Limiter
 - $f(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases}$
 - Use case: similar to step function but more efficient
 - Limitation: can not handle non-linear boundaries

Simple Neural Network Architecture

● Execution Example: Decision-Making in SLP

- Let's consider an example where an SLP predicts whether a student will pass (1) or fail (0) based on:
 - Study hours (x_1)
 - Previous exam score (x_2)
- **Step 1:** Define inputs, weights, and bias
 - $w_1 = 0.6, w_2 = 0.8$ (weights)
 - $b = -0.7$ (bias)
- Activation function: Step function $f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$
- Limitation: can not handle non-linear boundaries

Simple Neural Network Architecture

● Execution Example: Decision-Making in SLP

- Let's consider an example where an SLP predicts whether a student will pass (1) or fail (0) based on:
 - Study hours $x_1 = 2$
 - Previous exam score $x_2 = 3$
- **Step 2:** Compute network input z
 - $z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$
 - $z = (0.6 * 2) + (0.8 * 3) - 0.7 = 1.2 + 1.2 - 0.7 = 2.9$
- **Step 3:** Apply activation function
 - Since $z = 2.9$ is greater than 0, the activation function outputs:
 - $f(z) = 1$ (Pass)
 - Thus, the perceptron predicts that the student will pass

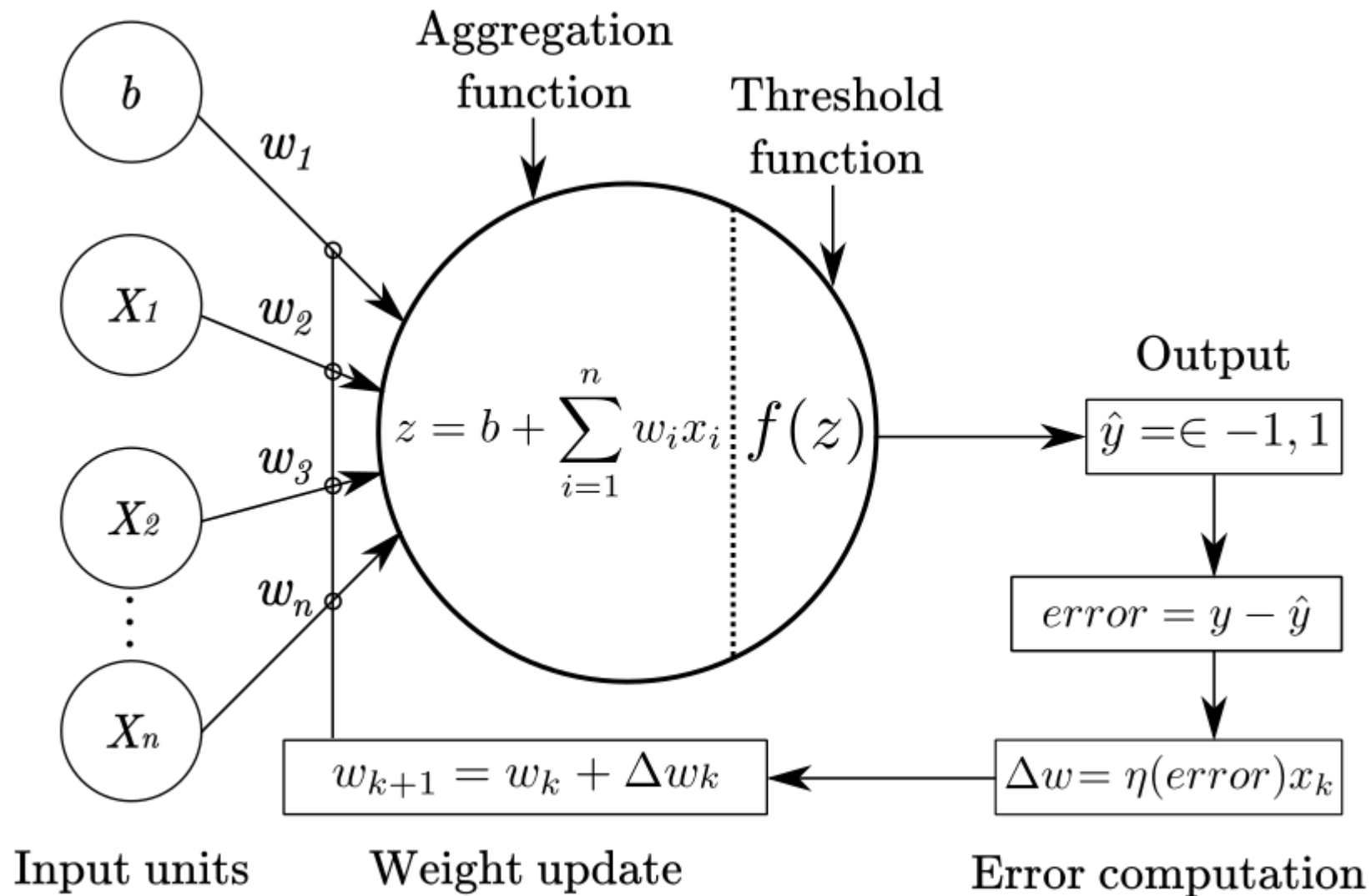
Simple Neural Network Architecture

- **Learning rule: weight update mechanism**
 - The perceptron learning rule is an iterative update mechanism that adjusts the weights based on classification errors
 - It ensures that the perceptron correctly classifies training samples in linearly separable problems
- **Weight update formula**
 - For a given training sample (X, y) , the weights W are updated as follow:
 - $w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$
 - $w_i \rightarrow$ Weights for feature x_i
 - $\eta \rightarrow$ Learning rate (step size for updates)
 - $y \rightarrow$ True label (actual class)
 - $\hat{y} \rightarrow$ Predicted output by perceptron
 - $x_i \rightarrow$ Input feature

Simple Neural Network Architecture

- Learning rule: weight update mechanism
- Bias update formula
 - The bias term b is also updated using the same rule:
 - $b \leftarrow b + \eta \cdot (y - \hat{y})$
- Explanation of the update rule
 - Correct classification: if $y = \hat{y}$, no weight update is needed
 - Misclassification: if $y \neq \hat{y}$, weights are updated:
 - If the perceptron predicted 0 but the actual label is 1, increase the weights to push the decision boundary in the right direction
 - If the perceptron predicted 1 but the actual label is 0, decrease weights to shift the boundary

Simple Neural Network Architecture



Simple Neural Network Architecture

- Learning rule: weight update mechanism
 - Input $X = (2,3)$
 - True class $y = 1$
 - Initial weights $w_1 = 0.5, w_2 = -0.4$
 - Bias $b = 0.2$
 - Learning rate $\eta = 0.1$
 - Predicted output $\hat{y} = 0$ (wrong prediction)
- Weight updates: since $y - \hat{y} = 1 - 0 = 1$, apply the update:
 - $W_1 \leftarrow 0.5 + (0.1 * 1 * 2) = 0.7$
 - $W_2 \leftarrow -0.4 + (0.1 * 1 * 3) = -0.1$
 - $b \leftarrow 0.2 + (0.1 * 1) = 0.3$
- After updating, the new parameters are:
 - $w_1 = 0.7, w_2 = -0.1, b = 0.3$
 - The perceptron moves the decision boundary to better classify the sample

Simple Neural Network Architecture

- When does training stop?
 - The perceptron updates weights until:
 - All training points are classified correctly (converges)
 - Or a maximum number of iteration is reached
 - If data is not linearly separable, the perceptron will never converge
 - If the data are is not linearly separable the perceptron will never converge
 - It keeps updating weights forever (or until a stopping condition is reached)
 - The decision boundary can not correctly separate the two classes
 - A Multi-Layer Perceptron (MLP) is needed to solve non-linearly separable problems (e.g., XOR problem)

Simple Neural Network Architecture

● Workshop 01

- Iris dataset is a well-known dataset in machine learning that consists of 150 samples of iris flowers, categorized into three species:
 - Setosa
 - Versicolor
 - Virginica
- Each flower is described by four features:
 - Sepal Length (cm)
 - Sepal width (cm)
 - Petal Length (cm)
 - Petal width (cm)
- Since SLP can only handle binary classification, we will simplify the problem by classifying
 - Setosa (Label: 0) vs. Versicolor (Label: 1)

● Workshop 02

● Load and Preprocess the data

- Load the Iris dataset from `sklearn.dataset`
- Select only two classes: `Setosa` and `Versicolor`
- Extract two features: `Petal length` and `Petal width`
- Normalize the data for better learning

● Train SLP

- Use the `Perceptron` model from `sklearn.linear_model`
- Train it using the selected features set

● Visualize decision boundary

- Plot the decision boundary learned by the perceptron
- Show classified points in two distinct colors

● Evaluate the model

- Compute accuracy on a test set
- Discuss whether SLP is a good choice for this problem (compare with other classifiers implemented in the previous workshop 01)

Simple Neural Network Architecture

- **Workshop 02**

- **Train SLP**

- Stochastic Gradient Descent (SGD) is the most commonly used optimization method for SLP
- `Scikit-Learn` Perceptron model supports only SGD optimizer (switch to `PyTorch` or `TensorFlow/Keras` to try other training algorithm like Adam, BGD, mini BGD... next workshop..)

● Workshop 02

- Load and Preprocess the data
 - Load the Iris dataset from `sklearn.dataset`
 - Select only two classes: `Setosa` and `Versicolor`
 - Extract two features: `Petal length` and `Petal width`
 - Normalize the data for better learning

Simple Neural Network Architecture

ModelPerformance.py X

SLP

SLP > iris

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import load_iris
4  from sklearn.linear_model import Perceptron
5  from sklearn.model_selection import train_test_split
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.metrics import classification_report
8
9  iris = load_iris()
10 X = iris.data[:, [2, 3]] # Petal Length & Petal Width
11 y = iris.target
12
13 binary_mask = (y == 0) | (y == 1)
14 X = X[binary_mask]
15 y = y[binary_mask]
16
17 y = np.where(y == 0, 0, 1)
18
19 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
20
21 scaler = StandardScaler()
22 X_train = scaler.fit_transform(X_train)
23 X_test = scaler.transform(X_test)
```

● Workshop 02

- Train SLP
 - Use the `Perceptron` model from `sklearn.linear_model`
 - Train it using the selected features set
- Visualize decision boundary
 - Plot the decision boundary learned by the perceptron
 - Show classified points in two distinct colors

Simple Neural Network Architecture

ModelPerformance.py

SLP

SLP > [x] perceptron

```
25 perceptron = Perceptron(max_iter=1000, eta0=0.01, random_state=42)
26 perceptron.fit(X_train, y_train)
27
28 accuracy = perceptron.score(X_test, y_test)
29 print(f"Model Accuracy: {accuracy * 100:.2f}%")
30
31 y_pred = perceptron.predict(X_test)
32 print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=["Setosa", "Versicolor"]))
33
34 def plot_decision_boundary(model, X, y):
35     x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
36     y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
37     xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
38                           np.linspace(y_min, y_max, 100))
39     Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
40     Z = Z.reshape(xx.shape)
41     plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.bwr)
42     plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k", cmap=plt.cm.bwr)
43     plt.xlabel("Petal Length (standardized)")
44     plt.ylabel("Petal Width (standardized)")
45     plt.title("Decision Boundary of Perceptron on Iris Dataset")
46     plt.show()
47 plot_decision_boundary(perceptron, X_train, y_train)
```

Simple Neural Network Architecture

● Workshop 02

● Evaluate the model

- Compute accuracy on a test set
- Discuss whether SLP is a good choice for this problem (compare with other classifiers implemented in the previous workshop 01)

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
Classification Report:				
	precision	recall	f1-score	support
Setosa	1.00	1.00	1.00	12
Versicolor	1.00	1.00	1.00	8
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

